

NAME :D.G.RAGHAVI

COURSE CODE : CSA1013

REG NO: 192521173

COURSE NAME : SOFTWARE

ENGINEERING

CO3-ASSESSMENT TOOL - 01

Title: AI-Powered Personalized Nutrition and Health Recommendation Platform

1. Problem Analysis

1.1 Problem Statement

Construction projects involve multiple activities such as material management, equipment utilization, worker safety, scheduling, and progress monitoring. Traditional construction monitoring depends heavily on manual inspections and periodic reports. This makes it difficult to obtain real-time information about project progress and safety conditions.

The proposed AI-Based Smart Construction Project Monitoring and Risk Assessment Platform uses AI, IoT sensors, cameras, drones, BIM data, and cloud-based technologies to continuously monitor construction activities.

The platform collects construction-site data and analyzes it to:

- Monitor construction progress.
- Detect safety violations.
- Predict possible project delays.
- Monitor equipment utilization.
- Optimize resource allocation.
- Assess construction risks.
- Generate project performance dashboards.
- Support management decision-making.

1.2 Project Objectives

The major objectives are:

1. Monitor construction progress continuously.
2. Detect safety violations using AI-based image analysis.
3. Predict project delays using historical and real-time data.
4. Optimize allocation of workers, materials, and equipment.
5. Track construction equipment utilization.
6. Generate project performance dashboards.

7. Identify and assess construction risks.
8. Improve overall project management efficiency.

1.3 Functional Requirements

Requirement	Description
User Management	Allows administrators, project managers, engineers, and safety officers to access the system.
Project Management	Allows users to create and monitor construction projects.
Site Monitoring	Collects data from cameras, drones, and IoT sensors.
Progress Tracking	Tracks percentage of project completion.
Safety Detection	Identifies safety violations such as missing helmets or unsafe zones.
Risk Assessment	Calculates project risks based on collected data.
Delay Prediction	Uses AI to predict possible schedule delays.
Resource Monitoring	Tracks workers, materials, and equipment.
Equipment Tracking	Monitors equipment utilization and availability.
Dashboard	Displays project KPIs, risks, progress, and alerts.
Notifications	Sends alerts when critical risks or safety violations are detected.
Report Generation	Generates project monitoring and risk reports.

1.4 Expected Outcomes

The expected outcomes include:

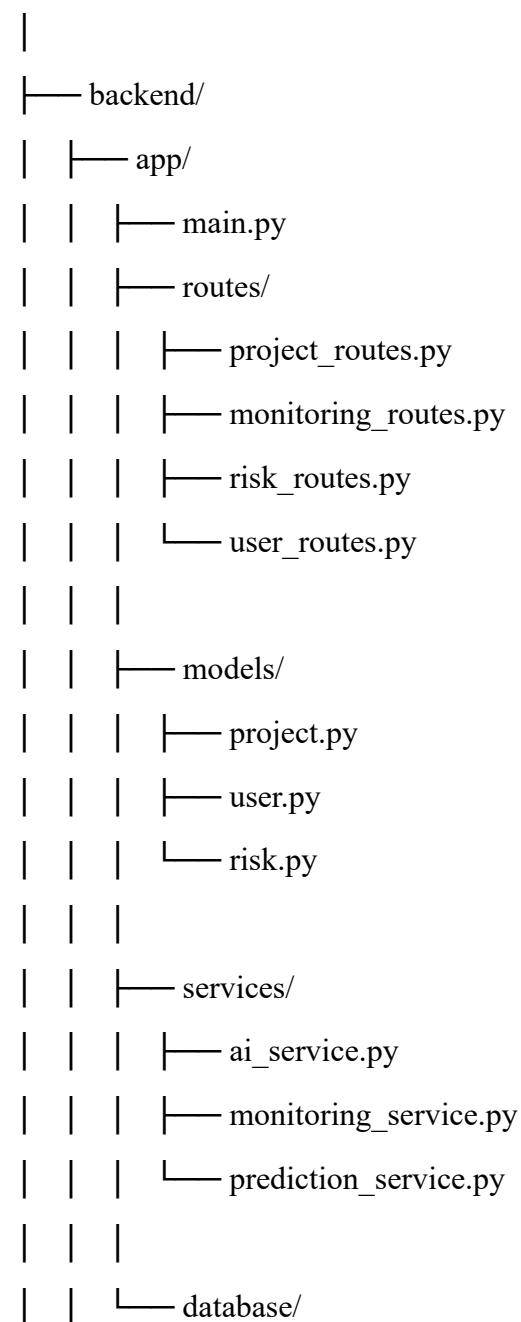
- Reduced construction delays.
- Improved workplace safety.
- Better resource utilization.
- Reduced project costs.

- More accurate project monitoring.
- Faster decision-making.
- Increased construction productivity.
- Improved overall project success rate.

2. Project Structure Design

A suitable repository structure is:

smart-construction-monitoring/



```
| | └── database.py
| |
| ├── requirements.txt
| └── Dockerfile
|
└── frontend/
    ├── src/
    |   ├── components/
    |   ├── pages/
    |   └── services/
    |   └── App.js
    ├── package.json
    └── Dockerfile
|
└── ai-model/
    ├── models/
    ├── datasets/
    ├── training/
    └── prediction.py
|
└── database/
    └── init.sql
|
└── tests/
    ├── test_project.py
    ├── test_risk.py
    └── test_monitoring.py
|
└── docs/
```

```
| |— architecture.png
| |— workflow.png
|
|— .gitignore
|— .env.example
|— README.md
|— Dockerfile
|— docker-compose.yml
```

Purpose of Major Folders

backend/

Contains the server-side application and REST APIs.

frontend/

Contains the user interface and project dashboard.

ai-model/

Contains AI models used for safety detection, risk assessment, and delay prediction.

database/

Contains database initialization scripts and schema definitions.

tests/

Contains automated test cases.

docs/

Contains architecture diagrams, workflow diagrams, and project documentation.

Important Files

main.py – Entry point for the backend application.

requirements.txt – Contains Python dependencies.

Dockerfile – Defines how the application container is built.

docker-compose.yml – Defines and manages multiple application containers.

.gitignore – Specifies files that should not be uploaded to Git.

.env.example – Provides sample environment variables without exposing sensitive values.

README.md – Contains project information, installation instructions, and usage instructions.

3. Git Repository Initialization

Git is used to track changes to the project source code and enable multiple developers to collaborate.

Git Initialization Steps

First, create the project folder:

```
mkdir smart-construction-monitoring
```

```
cd smart-construction-monitoring
```

Initialize Git:

```
git init
```

Check repository status:

```
git status
```

Add project files:

```
git add .
```

Create the first commit:

```
git commit -m "Initial project setup"
```

Connect the local repository to GitHub:

```
git remote add origin <repository-url>
```

Push the project:

```
git branch -M main
```

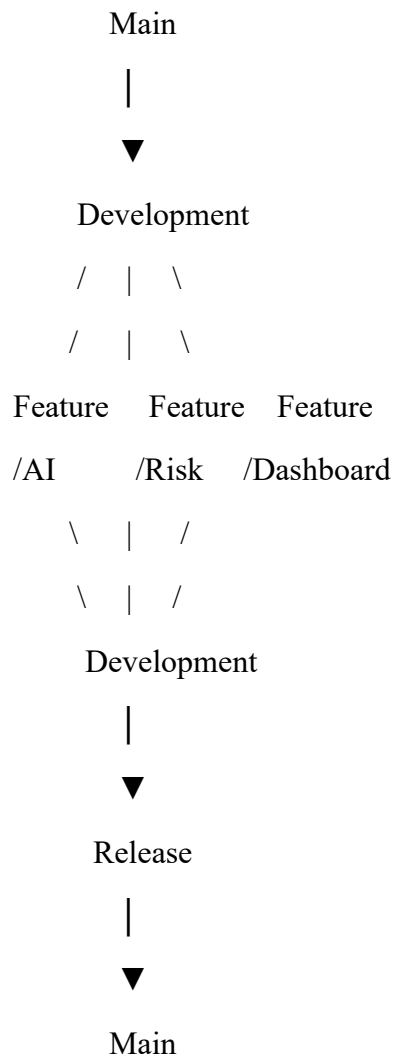
```
git push -u origin main
```

Essential Git Files

File	Purpose
.git/	Stores Git repository information and history.
.gitignore	Prevents unnecessary or sensitive files from being tracked.
README.md	Provides project documentation.
.env	Stores environment-specific configuration. It should normally not be committed.
.env.example	Shows required environment variables without exposing secrets.

4. Git Branching Strategy

The project uses a Git Flow-inspired branching strategy.



Hotfix —————> Main

Main Branch

Contains stable and production-ready code.

Development Branch

Contains the latest integrated development code.

Feature Branch

Used to develop individual functionalities.

Examples:

feature/ai-risk-analysis

feature/safety-detection

feature/dashboard

feature/equipment-monitoring

Release Branch

Used for final testing and preparation before production deployment.

Justification

This branching model is suitable because the project contains several independent modules such as AI analysis, dashboards, monitoring, and risk assessment.

Developers can work independently on feature branches without affecting the stable application.

6. Commit History Analysis

Meaningful commit messages should clearly describe what was changed.

Example commit history:

a91f21d Add construction project dashboard

b82e34a Implement AI risk prediction service

c73d45f Add safety violation detection API

d64e56a Add equipment monitoring module

e55f67b Create project database schema

f46a78c Add backend REST API

g37b89d Initial project setup

Good Commit Message Practices

Commit messages should:

- Be short and meaningful.
- Describe one logical change.
- Use action-oriented language.
- Avoid vague messages such as "changes" or "update".
- Make future debugging easier.

Examples

Good:

Add AI-based delay prediction

Fix equipment utilization calculation

Add safety violation API

Update project dashboard

Resolve database connection error

7. Docker Environment Design

Docker is suitable for this project because the platform contains multiple components with different dependencies.

The application can contain:

Frontend

|



Backend API

|

|—————▶ AI Service

|

|—————▶ Database

|

|—————▶ Monitoring Service

Components Requiring Containerization

1. Frontend
2. Backend API
3. AI service
4. Database
5. Optional monitoring/processing service

Benefits of Docker

Docker provides:

- Consistent development environments.
- Dependency isolation.

- Easy deployment.
- Application portability.
- Easier scaling.
- Simplified environment configuration.

For example, developers do not need to manually install every Python package, Node.js dependency, or database configuration on their computers

11. Benefits of Git and Docker

Area	Git	Docker
Collaboration	Multiple developers can work on branches.	Developers use consistent environments.
Version Management	Tracks every source-code change.	Tracks application/container versions through images.
Portability	Repository can be cloned anywhere.	Containers can run on different systems.
Deployment	Code can be integrated and released systematically.	Application can be deployed consistently.
Scalability	Supports team-based development.	Multiple application containers can be scaled.
Maintainability	Commit history makes changes traceable.	Dependencies are isolated.
Reliability	Changes can be reverted.	Environment differences are reduced.
Testing	Branches allow isolated testing.	Testing can occur in reproducible containers.

12. Challenges and Improvements

12.1 Challenges Encountered

1. AI Model Integration

Integrating AI models with the backend can be challenging because AI models may require large libraries and computational resources.

2. Multiple Data Sources

Drone images, camera feeds, IoT sensor data, and BIM information can have different formats.

3. Docker Dependency Issues

Different services may require different versions of Python, Node.js, or other dependencies.

4. Database Connectivity

The backend must correctly communicate with the database container.

5. Git Merge Conflicts

When multiple developers modify the same files, merge conflicts can occur.

6. Large AI Models

Large computer vision models may increase Docker image size and deployment time.

7. Real-Time Processing

Processing continuous camera and IoT data requires efficient computing and communication.

12.2 Possible Improvements

Future versions can include:

1. Real-time drone video analysis.
2. Advanced computer vision for PPE detection.
3. BIM integration.
4. Predictive maintenance for construction equipment.
5. Real-time IoT sensor monitoring.
6. Cloud deployment.
7. Kubernetes-based container orchestration.
8. Automatic CI/CD pipelines.
9. Advanced AI-based schedule prediction.
10. Mobile application support.
11. Automated alert and notification systems.
12. Historical project analytics.

Conclusion

The **AI-Based Smart Construction Project Monitoring and Risk Assessment Platform** provides a technology-driven solution for improving construction project management. AI can be used to identify safety violations, predict delays, assess risks, and analyze construction progress, while IoT devices, drones, cameras, and BIM provide continuous project data.

Git provides structured version control, collaboration, branching, and change tracking. Docker provides a consistent and portable environment for running the frontend, backend, AI services, and database. Docker Compose simplifies the management of these services.

Together, **Git and Docker support a reliable, maintainable, scalable, and collaborative development and deployment workflow** for the proposed smart construction platform.